

09 · APIs e integraciones

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 09-apis-and-integrations.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. La API HTTP del panel
2. Ejemplos de uso
3. Contrato de datos: `/metrics` en formato Prometheus
4. Integraciones salientes
5. Adaptador opcional de visión multimodal (no usado por ningún flow)
6. Interfaces de plataforma (Windows)
7. Extensión por terceros: entry points
8. Lo que NO existe

Endpoints internos, contratos de datos, integraciones salientes, dependencias de plataforma y el adaptador de visión multimodal que existe pero ningún flow usa. Todos los ejemplos usan valores ficticios y dominios reservados.

1. La API HTTP del panel

Aspecto	Valor
Servidor	<code>ThreadingHTTPServer</code> de la biblioteca estándar
Bind por defecto	<code>127.0.0.1:8787</code> (<code>app/server.py::run_server</code>)
Protocolo	HTTP/1.1 sin TLS
Formatos	<code>application/json; charset=utf-8, text/html; charset=utf-8, text/plain; version=0.0.4</code>
Versionado	Ninguno. No hay <code>/v1/</code> , ni cabecera de versión
CORS	No se emite ninguna cabecera CORS. El navegador bloquea cualquier lectura cross-origin
Documentación de API	NO DOCUMENTADO EN EL REPOSITORIO: no hay OpenAPI ni Swagger

Sin TLS y sin CORS, la superficie efectiva es la máquina local. La única cabecera de seguridad que se emite es `X-Content-Type-Options: nosniff`, y solo en `/file`.

1.1 Endpoints GET

Método	Ruta	Devuelve	Auth
GET	<code>/</code>	HTML del panel de 3 pestañas	[NO]
GET	<code>/healthz</code>	<code>{"status": "ok"}</code>	[NO]
GET	<code>/metrics</code>	Texto Prometheus	[NO]
GET	<code>/api/metrics</code>	<code>{"overview": {...}, "by_flow": [{...}]}</code>	[NO]
GET	<code>/metrics/dashboard</code>	HTML del dashboard	[NO]
GET	<code>/api/flows</code>	Array de flows	[NO]
GET	<code>/api/runs?flow_id=&limit=50</code>	Array de corridas	[NO]
GET	<code>/api/runs/<run_id>/status</code>	Estado paso a paso	[NO]
GET	<code>/flow/<folder></code>	HTML de la ficha	[NO]
GET	<code>/flow/<folder>/config</code>	HTML del editor de contexto	[NO]
GET	<code>/flow/<folder>/history</code>	HTML del histórico del flow	[NO]
GET	<code>/run/<flow_id>/<run_id></code>	HTML del detalle	[NO]
GET	<code>/file?path=<relativa></code>	Bytes del archivo	[NO]

Ninguna lectura exige autenticación, ni siquiera con `AUTOMA_PANEL_TOKEN` definido: `do_GET` no llama a `_authorize_mutation`. Ver [11 · Seguridad](#).

1.2 Endpoints POST

Método	Ruta	Cuerpo	Devuelve	Auth
POST	/api/run/<folder>	{"context_overrides": {...}} opcional	{ok, run_id, status, flow_id}	_authorize_mutation
POST	/api/hook/<folder>	Ignorado	{ok, run_id, status, flow_id}	AUTOMA_WEBHOOK_TOKEN
POST	/api/form/submit	JSON libre	{ok, saved_path}	_authorize_mutation
POST	/run?flow=<folder>	Formulario	303 → /run/<flow_id>/<run_id>	_authorize_mutation
POST	/flow/<folder>/config	config_json=<JSON>	HTML	_authorize_mutation
POST	/flow/<folder>/schedule	enabled, interval_seconds, cron_expression	303 → /#schedule	_authorize_mutation

1.3 Códigos de estado

Código	Cuándo
200	Éxito
303	Redirección tras un POST de formulario
400	folder que no pasa _safe_folder, JSON malformado, cuerpo vacío en /api/form/submit, error al guardar config o schedule
401	No autorizado: token inválido, Host no loopback, Origin/Referer incoherentes, o AUTOMA_WEBHOOK_TOKEN sin definir
404	Flow inexistente, corrida inexistente, ruta no servida, archivo no encontrado en /file
409	Flow marcado preview: true o con archivo .disabled
415	/file con extensión de la lista bloqueada
500	FlowExecutionError en la ejecución síncrona (/run, /api/hook) o al instanciar el orquestador

2. Ejemplos de uso

Disparar un flow y hacer polling

```
# 1. Disparo asíncrono: devuelve de inmediato
curl -s -X POST http://127.0.0.1:8787/api/run/05_system_healthcheck \
  -H 'Content-Type: application/json' \
  -d '{}'
```

```
{
  "ok": true,
  "run_id": "20260827T143052123456Z",
  "status": "running",
  "flow_id": "system_healthcheck"
}
```

```
# 2. Polling del estado paso a paso
curl -s http://127.0.0.1:8787/api/runs/20260827T143052123456Z/status
```

```
{
  "ok": true,
  "run_id": "20260827T143052123456Z",
  "flow_id": "system_healthcheck",
  "flow_folder": "05_system_healthcheck",
  "flow_name": "Healthcheck del sistema",
  "status": "completed",
  "steps": [
    {"step_id": "take_snapshot", "action": "system.snapshot_system", "status": "success", "attempt": 1, "duration_": 0.001},
    {"step_id": "evaluate_snapshot", "action": "rules.evaluate", "status": "success", "attempt": 1, "duration_": 0.001},
    {"step_id": "write_snapshot", "action": "filesystem.write_json", "status": "success", "attempt": 1, "duration_": 0.001}
  ],
  "duration_seconds": 0.4312,
  "started_at": "2026-08-27T14:30:52.200000+00:00",
  "finished_at": "2026-08-27T14:30:52.631200+00:00",
  "error": null
}
```

Las 11 claves de primer nivel son exactamente las que devuelve `app/server.py:_run_status_payload`, que combina la fila de `runs`, los registros de `steps` y la lista de pasos del manifest. Un paso sin registro aparece como `running` si es el primero pendiente de una corrida viva, `pending` para los siguientes de esa misma corrida, y `not_taken` si la corrida ya terminó —porque entonces significa «rama no tomada», no «pendiente».

Disparar con override de contexto

```
curl -s -X POST http://127.0.0.1:8787/api/run/03_folder_inventory \
  -H 'Content-Type: application/json' \
  -d '{"context_overrides": {"path_override": "C:/Users/ejemplo/Documentos"}}'
```

El override reemplaza claves de **primer nivel**; no hay fusión profunda.

Webhook entrante

```
# Requiere AUTOMA_WEBHOOK_TOKEN definido en el entorno del panel
curl -s -X POST http://127.0.0.1:8787/api/hook/05_system_healthcheck \
  -H 'X-Automa-Token: TOKEN-DE-EJEMPLO-NO-REAL'
```

Diferencia crítica con `/api/run`: el webhook es **síncrono**. Ejecuta el flow completo y responde al terminar. Una petición contra un flow largo mantiene la conexión abierta hasta el final, y un timeout del cliente no cancela la corrida.

```
{"ok": true, "run_id": "20260827T143100987654Z", "status": "completed", "flow_id": "system_healthcheck"}
```

Sin token configurado:

```
{"ok": false, "error": "token inválido o AUTOMA_WEBHOOK_TOKEN no configurado"}
```

Panel con token

```
export AUTOMA_PANEL_TOKEN='TOKEN-DE-EJEMPLO-NO-REAL' # en el proceso del panel
curl -s -X POST http://127.0.0.1:8787/api/run/05_system_healthcheck \
  -H 'X-Automa-Token: TOKEN-DE-EJEMPLO-NO-REAL' -d '{}'
```

La comparación se hace con `hmac.compare_digest`, en tiempo constante (cierra CWE-208).

Programar un flow

```
# Cada 15 minutos, en punto (recordar: los campos van en UTC)
curl -s -X POST 'http://127.0.0.1:8787/flow/05_system_healthcheck/schedule' \
  -d 'enabled=on' --data-urlencode 'cron_expression=*/15 * * * *'

# 0 por intervalo simple, en segundos
```

```
curl -s -X POST 'http://127.0.0.1:8787/flow/05_system_healthcheck/schedule' \
-d 'enabled=on' -d 'interval_seconds=900'
```

`set_schedule` prioriza `cron_expression` si viene; el panel envía `interval_seconds` solo cuando no hay cron.

Servir un archivo de salida

```
curl -s -o captura.png 'http://127.0.0.1:8787/file?path=output/screenshots/desktop_clean_20260827_143052.png'
```

`/file` acepta **solo rutas relativas** a la raíz del proyecto y rechaza `.html`, `.htm`, `.xhtml`, `.xml`, `.svg`, `.js`, `.mjs` y `.css` con 415. Los `.png`, `.json`, `.csv`, `.md`, `.log` y `.jsonl` **sí se sirven**.

3. Contrato de datos: `/metrics` en formato Prometheus

`engine/metrics.py::prometheus_text` emite un puñado de series. Ejemplo real de la forma:

```
# HELP flujo_runs_total Total de corridas por estado.
# TYPE flujo_runs_total counter
flujo_runs_total{status="completed"} 148
flujo_runs_total{status="failed"} 3
# HELP flujo_run_duration_seconds_avg Duración promedio histórica.
# TYPE flujo_run_duration_seconds_avg gauge
flujo_run_duration_seconds_avg 0.8123456
flujo_runs_window_completed 96
flujo_runs_window_failed 2
```

Observaciones sobre el contrato:

- El prefijo es `flujo_`, no `automa_`. Es un resto del nombre anterior del proyecto y quedaría feo cambiarlo sin romper dashboards existentes. Referencia histórica, no error.
- `flujo_runs_window_completed` y `flujo_runs_window_failed` se emiten **sin # HELP ni # TYPE**. Prometheus lo tolera, pero es una inconsistencia del formato.
- La ventana es de 200 corridas, fija (`overview(window_runs=200)`).
- El endpoint **no exige autenticación**. Un scraper en la red local podría leerlo si el panel se expusiera fuera de loopback.

`/api/metrics` devuelve el mismo material en JSON, con más detalle: `slowest_actions`, `retries_top_actions`, `failed_top_actions` y el desglose `by_flow`.

4. Integraciones salientes

4.1 Chromium vía Playwright

Aspecto	Detalle
Acciones	<code>browser.capture_page</code> , <code>browser.fill_form</code> , <code>browser.extract_content</code> , <code>browser.crawl_site</code>
Instalación	<code>pip install playwright && python -m playwright install chromium</code>
No declarada	<code>playwright</code> no está en <code>pyproject.toml</code> ni en <code>requirements.txt</code>
Modo	Headless salvo <code>browser.fill_form</code> , que por defecto abre ventana visible
User-Agent	<code>automa-pc</code> en <code>extract_content</code> y <code>crawl_site</code> ; el de Chromium por defecto en los otros dos
Timeout	<code>timeout_seconds=30.0</code> por defecto, aplicado con <code>page.set_default_timeout</code>

Degradación	<code>RuntimeError</code> con el comando de instalación exacto
-------------	--

`_to_url` acepta `http://`, `https://`, `file://` o una ruta local a un `.html`. Una ruta inexistente que no sea URL levanta `FileNotFoundError`.

4.2 robots.txt

`actions/browser_extract.py::RobotsCache` consulta `<esquema>://<host>/robots.txt` una vez por host, con `User-Agent: automa-pc` y timeout de 5 s.

Situación	Comportamiento
<code>robots.txt</code> legible	Se aplica <code>RobotFileParser.can_fetch</code>
Estado ≥ 400	Se permite; <code>checked=True</code>
Error de red	Se permite ; <code>checked=False</code> , reportado en <code>robots_checked_hosts</code>
Esquema no http(s) (por ejemplo <code>file://</code>)	Se permite sin consultar

El fallo es permisivo pero **declarado**: el docstring lo dice y el reporte del flow 22 lo expone. Es la decisión honesta.

4.3 Verificación de enlaces

`http.check_urls` hace `HEAD` con redirects y, ante `405` o `501`, reintenta con `GET` en modo `stream` cerrando la respuesta sin descargar el cuerpo. `file://` se verifica por existencia en disco. `mailto:` y `tel:` se marcan `skipped`.

Sin control de reintentos ni de concurrencia: las peticiones son secuenciales, con `delay_seconds` opcional entre ellas. Un `max_urls` de 100 contra un servidor lento con timeout de 10 s puede tardar hasta 1 000 segundos. **INFERENCIA**.

4.4 Notificaciones salientes

`actions/notify.py::send_notification` con `backend: "webhook"`:

```
POST <target>
Content-Type: application/json
Authorization: Bearer <token resuelto> ← solo si se pasó token

{
  "text": "[!] Cambio detectado en https://ejemplo.example/pagina · hash a3f5...",
  "timestamp": "2026-08-27T14:30:52.123456+00:00"
}
```

Compatible con Slack y Discord si el endpoint acepta `{"text": "..."}`. El campo `extra` del manifest se fusiona en el cuerpo.

Resolución de secretos: un `token` que empiece por `@secret:` se resuelve con `engine.secrets.get_secret`. Ejemplo de manifest:

```
{
  "action": "notify.send",
  "params": {
    "message": "...",
    "backend": "webhook",
    "target": "https://hooks.ejemplo.example/servicios/XXXX",
    "token": "@secret:NOTIFY_TOKEN"
  }
}
```

El token nunca se devuelve en el resultado. `record` contiene `backend`, `message`, `timestamp`, `sent`, `target` y `status_code` — no el token. Es correcto y deliberado, porque ese resultado acaba en `runs.context_json`.

Sin reintentos ni cola: un webhook caído produce una excepción de `requests` que el orquestador convierte en fallo del paso. La única política de reintento disponible es el campo `retries` del paso en el manifest.

Ningún flow usa backend: "webhook" por defecto. Los flows 23 y 26 traen `notify_backend: "file"` en su `context.example.json`.

4.5 Tesseract OCR

Integración con un **binario externo**, no con un servicio. `pytesseract` invoca `tesseract` en el sistema. `OCRImageAnalyzer._tesseract_binary_available` lo busca en el PATH y, además, en `C:/Program Files/Tesseract-OCR/tesseract.exe` y su equivalente (`x86`), que es donde el instalador de Windows lo deja sin añadirlo al PATH.

Si falta, devuelve un payload válido con `status: "unavailable"`, `reason` (`pytesseract_missing` o `tesseract_binary_missing`) y una `summary` con el comando de instalación por sistema operativo. **El flow no falla.**

5. Adaptador opcional de visión multimodal (no usado por ningún flow)

Este es el único punto del repositorio donde existe código capaz de llamar a un modelo de IA. Conviene documentarlo con precisión, porque el producto se presenta —correctamente— como determinista y sin LLM.

5.1 Qué existe

`plugins/analyzers/vision_model_analyzer.py::VisionModelAnalyzer.analyze` soporta tres proveedores:

provider	Endpoint	Autenticación	Red
mock (por defecto)	—	—	[NO]
openai_compatible	<endpoint>/chat/completions	Bearer desde <code>api_key</code> , <code>api_key_env</code> o <code>OPENAI_API_KEY</code>	[OK]
ollama	<endpoint o <code>http://127.0.0.1:11434>/api/chat</code>	Ninguna	[OK] (local por defecto)

`mock` no llama a nada: abre la imagen con Pillow, calcula brillo medio y RGB, y devuelve `target_found: False`, `confidence: 0.0`. Su propia `summary` lo declara: «Visión mock completada sin OCR ni proveedor externo [...] no identifica texto real».

Los dos proveedores reales codifican la imagen en base64 (data URL en el caso OpenAI-compatible), piden una respuesta **solo JSON** con `summary`, `target_found`, `confidence`, `target_bbox` y `visible_text`, y usan `_extract_json_object` para recuperar el objeto aunque venga envuelto en texto. `temperature: 0` en el caso OpenAI-compatible. Sin endpoint o sin modelo, levantan `RuntimeError`.

5.2 Por qué es inalcanzable desde el catálogo

Cadena de invocación completa, verificada con búsqueda en todo el repositorio:

```
VisionModelAnalyzer.analyze
  ← llamado SOLO por actions/vision.py::inspect_screen_target
    ← registrado como acción 'vision.inspect_screen_target'
      ← usada por 0 de los 27 manifests
```

Comprobación reproducible:

```
grep -l "inspect_screen_target" flows/*/manifest.json      # sin resultados
```

Conclusión verificable: con el catálogo tal y como se distribuye, **ningún flow puede provocar una llamada a un proveedor de IA**. La instancia `VISION_ANALYZER = VisionModelAnalyzer()` se crea al importar `actions/vision.py`, pero su constructor no hace nada y ningún método se ejecuta sin pasar por `inspect_screen_target`.

Activarlo exigiría **escribir un flow nuevo** que declarara esa acción con `vision_provider` distinto de `mock` y un `vision_endpoint`. Es una decisión explícita del operador, no un comportamiento por defecto.

5.3 Riesgos si alguien lo activara

Riesgo	Detalle
Exfiltración de pantalla	La imagen enviada es una captura del escritorio, codificada íntegra en base64

Clave en el manifest	<code>vision_api_key</code> es un parámetro de la acción: escribirla ahí la dejaría en <code>steps.params_json</code> en claro
Sin validación del endpoint	Cualquier URL es aceptada
Determinismo perdido	La decisión <code>click</code> / <code>recover</code> pasaría a depender de un modelo

La forma segura sería `vision_api_key_env` con el nombre de una variable, nunca `vision_api_key` con el valor. **NÓ DOCUMENTADO EN EL REPOSITORIO**: no hay guía sobre esto.

5.4 Los stubs de `decision/`

`decision/optional_ai.py::suggest_step_order` y `decision/rules.py::prioritize_steps` devuelven la lista de pasos sin tocarla y **nadie los importa**. La docstring del primero documenta la intención de diseño, que sigue siendo válida aunque el código no exista:

«Stub para futura integración IA. La IA solo debe sugerir orden o prioridad, nunca reemplazar la ejecución del motor.»

Es una decisión arquitectónica registrada. El código muerto asociado está en [15 · Riesgos y deuda técnica](#).

6. Interfaces de plataforma (Windows)

No son APIs de red, pero son las integraciones que definen el producto.

Interfaz	Vía	Acciones	Qué falla sin ella
Captura de framebuffer	<code>mss</code> , respaldo <code>Pillow.ImageGrab</code>	<code>screen.*</code>	<code>RuntimeError</code> explícito
Teclado y ratón	<code>pyautogui</code>	<code>ui.hotkey</code> , <code>type_text</code> , <code>click</code> , <code>click_bbox</code>	<code>RuntimeError</code> con instrucción
Ventana en foco	<code>pygetwindow</code>	<code>screen.capture_active_window</code>	<code>RuntimeError</code> con motivo
Portapapeles	<code>pyperclip</code>	<code>system.read_clipboard</code>	<code>available: False</code> con razón, sin excepción
PowerShell	<code>subprocess</code>	<code>system.run_powershell</code>	Depende del SO
Lanzador de procesos	<code>subprocess.Popen</code> + <code>shlex</code>	<code>ui.launch_process</code>	—
Navegador del sistema	<code>webbrowser</code>	<code>ui.open_url</code> , <code>open_file_in_browser</code>	—
Ventana nativa	<code>pywebview</code> (<code>edgechromium</code> en Windows)	<code>automa-desktop</code>	Código de salida 2
CPU, RAM, disco, procesos	<code>psutil</code>	<code>system.*</code>	—

URIs `ms-settings:` — el flow 11 usa `ui.open_url` con URIs del tipo `ms-settings:network`. `webbrowser.open` delega en el manejador de protocolo de Windows. Es una integración con el shell del sistema operativo, no con la web.

7. Extensión por terceros: entry points

`LazyActionRegistry._maybe_load_entry_points` descubre el grupo `automa.actions`:

```
# pyproject.toml de un paquete de terceros
[project.entry-points."automa.actions"]
"miempresa.exportar_sap" = "mi_paquete.acciones:exportar_sap"
```

Reglas del mecanismo, verificadas en el código:

- Los entry points se cargan **una sola vez**, de forma perezosa, cuando se pide un nombre desconocido o se llama a `keys()`.
- Se usa `setdefault`: **una acción interna nunca puede ser sobrescrita** por un paquete externo. Es una decisión de seguridad relevante.
- La función debe aceptar los parámetros como argumentos por nombre y devolver un `dict` serializable a JSON.

Contrato completo en `docs/EXTENSION.md`.

Discrepancia verificada: `_BUILT_IN_ACTIONS` declara 36 acciones y `[project.entry-points."automa.actions"]` del `pyproject.toml` solo 31. Faltan `browser.capture_page`, `browser.crawl_site`, `browser.extract_content`, `browser.fill_form` y `http.check_urls`. No rompe el runtime —el diccionario interno se consulta primero— pero un paquete externo que inspeccione el grupo verá un catálogo incompleto. Registrado en 15.

8. Lo que NO existe

Verificado con búsqueda en todo el repositorio:

No existe	Comprobación
Cliente de base de datos remota	Solo <code>sqlite3</code>
Cola de mensajes, broker, <code>celery</code>	Sin dependencias de ese tipo
Autenticación OAuth / OIDC / SSO	Sin bibliotecas de auth
Telemetría o <i>analytics</i> del producto	Sin cliente saliente propio
API pública versionada	Sin <code>/v1/</code> , sin OpenAPI
Servidor gRPC o WebSocket	Solo HTTP/1.1
Cliente de correo	Sin <code>smtplib</code> en el código de producción
Integración con servicios en la nube	Sin SDK de ningún proveedor
Actualización automática	El instalador no incluye <i>updater</i>

```
# Comando que respalda la afirmación de ausencia de red no declarada
grep -rn "requests\.\|urllib.request\|http.client\|socket\." --include="*.py" \
  actions/ engine/ app/ plugins/ decision/ | grep -v "^Binary"
```

Devuelve únicamente: `actions/http_actions.py`, `actions/notify.py`, `actions/browser_extract.py` (`RobotsCache`), `plugins/analyzers/vision_model_analyzer.py` y `app/desktop.py` (`socket.create_connection` contra `127.0.0.1`, para esperar al servidor local). No hay ninguna otra salida de red en el sistema.

Documentos relacionados: 05 · Referencia técnica · 08 · Flujo de datos · 10 · Configuración · 11 · Seguridad · 13 · Despliegue y operación